

Large language models for partial differential equation workflows

Han Wan¹, Rui Zhang¹, Hao Sun¹

¹Gaoling School of Artificial Intelligence, Renmin University of China, Beijing 100872, China

Abstract

Partial differential equations (PDEs) become actionable in science and engineering not as isolated formulae, but as executable workflows that connect modelling assumptions, governing equations, numerical solvers, diagnostics, and decisions. Large language models (LLMs) are beginning to support such workflows by linking natural language, symbolic mathematics, code, solver outputs, and feedback. Here we examine recent advances in LLM-assisted PDE research across three stages: the discovery and formulation of governing models, the generation and revision of executable numerical solvers, and the use of simulation feedback to support control, design, and optimization. Across these stages, current systems act primarily as workflow-level interfaces. **Despite this progress, the field remains limited by the scarcity of high-quality datasets and benchmarks, especially for knowledge discovery and real-world applications, where expert annotation, executable problem construction, and task-level feedback require substantial domain effort.** A further challenge is the persistent gap between simulation-based results and real-world scientific and engineering systems, which limits the direct transfer of numerical simulations, control policies, and optimized designs to practical settings. **These challenges make LLM-assisted PDE workflows a critical testbed for developing scientific AI systems that can connect language, computation, physical constraints, and real-world decision-making.**

Keywords: Large Language Models, Partial Differential Equations, Scientific Workflow Automation, Executable Workflows, Lifecycle Taxonomy

1 Introduction

Partial differential equations (PDEs) describe how continuous systems evolve in space and time and provide compact mathematical representations of physical mechanisms and multiscale dynamics across diverse domains, making them a central mathematical language of science and engineering [1, 2]. Yet the scientific value of PDEs does not reside in their mathematical form alone, but in how they are formulated or discovered, solved, and applied to concrete scientific and engineering problems. Only when PDEs are brought into practical workflows through scientific computing can their predictive capability, interpretability, and task-level value in real-world applications ultimately be assessed [3].

Traditional PDE scientific computing has established much of the methodological foundation for this workflow, but its major stages have long depended on expert judgement and manual coordination (Fig. 1a). Governing equations have traditionally been established through an expert-driven process in which candidate hypotheses are proposed from first principles, physical constraints, mechanistic assumptions, and observations, and then tested and refined against empirical evidence,

with system identification providing complementary means of connecting observations to candidate model structures [4]. These governing equations then form the basis for numerical computation, where finite-difference and finite-volume methods are widely used for time-dependent problems and conservation laws, finite-element methods accommodate variational formulations and complex geometries, and spectral methods provide high-order approximations [5–8]. Their reliability is underpinned by mature analyses of stability, convergence, accuracy, and, where appropriate, conservation, yet practical performance still depends on expert choices in geometry representation, meshing, discretization, and solver configuration [9, 10]. PDE solutions are further embedded in optimal control, PDE-constrained optimization, flow control, and topology optimization to support scientific decision-making and engineering design [11–14]. However, such applications require large numbers of repeated PDE solves and substantial human intervention, leading to high computational and operational costs in complex settings [3, 12–14].

Data-driven methods and deep-learning approaches have enhanced several local components across model discovery, numerical solving, and downstream applications. In model discovery and formulation, sparse regression and related data-driven methods infer governing equations or candidate terms from measurements [15, 16], closure learning and coarse-grained modelling support reduced descriptions of fluid and multiscale systems [17–19], and physics-informed frameworks can be used to identify unknown parameters within prescribed model structures [20]. In numerical solving, learned discretizations and machine-learning-accelerated solvers can replace or augment components of traditional simulation pipelines [21, 22], while neural operators accelerate the approximation of solution maps [23–25]. Physics-informed learning incorporates physical constraints into training objectives [20, 26, 27], whereas physics-encoded architectures introduce inductive structure directly into model design [28, 29]. In downstream applications, deep reinforcement learning has been used for active flow control [30], differentiable physics has been used for PDE control [31], and learned simulators or graph network simulators have supported shape and design optimization tasks [32, 33]. However, these advances primarily improve individual components of PDE workflows and usually remain tied to particular stages, task classes, or model structures. They do not by themselves provide a mechanism for coordinating model discovery, numerical execution, solver feedback, and downstream objectives across a complete workflow.

LLMs enter PDE scientific computing at a different level of abstraction. By drawing on textbooks, research papers, solver documentation, and code repositories, they can translate accumulated scientific and computational knowledge into workflow-level assistance for constructing executable computational pipelines. This does not mean that LLMs replace numerical solvers or enable fully autonomous PDE science; rather, their role is to connect heterogeneous forms of knowledge, computation, and feedback across the workflow. General LLM capabilities in code generation [34], structured reasoning [35], and tool use [36] provide some of the technical foundations for this workflow-level role. In the **Discovery Stage**, emerging PDE-oriented LLM systems explore equation discovery, hypothesis selection, and the formalization of PDE specifications from data, text, or partial symbolic descriptions [37, 38]. In the **Solving Stage**, LLM-based systems have been used to generate solver code [39], configure CFD workflows [40], and revise executable components in response to solver feedback [41]. In the **Optimization Stage**, early studies have explored LLM assistance in formulating PDE control problems [42] and engineering design objectives such as parametric shape optimization [43]. By linking scientific knowledge, executable tools, and iterative feedback, these systems point toward a broader shift from isolated task automation to integrated scientific workflow automation [44, 45]. For PDE-centered scientific computing, realizing this potential requires outputs that are executable, inspectable, and consistent with numerical requirements and physical assumptions.

This Review therefore organizes LLMs for PDE scientific computing into three categories: **LLMs**

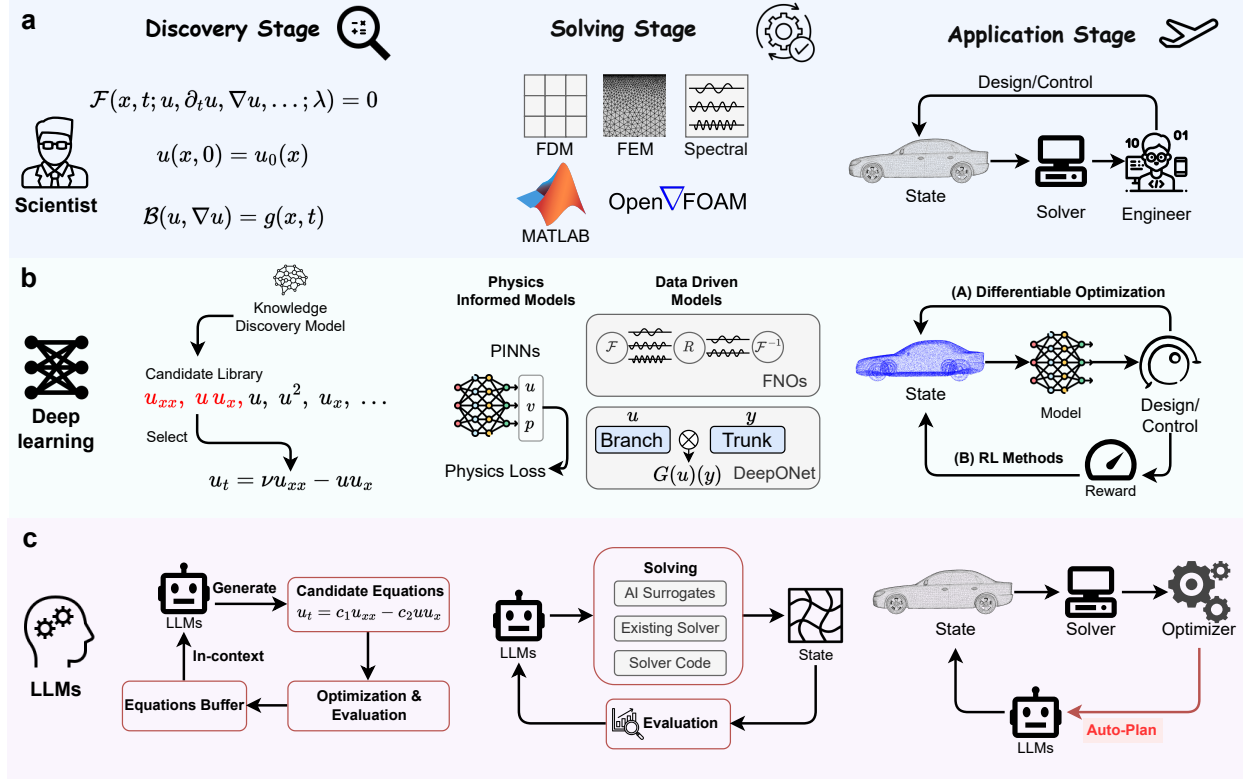


Figure 1: Comparison of traditional, deep-learning, and LLM-assisted workflows for PDE scientific computing. The workflow is organized into three stages: **Discovery**, **Solving**, and **Application**. (a) Traditional PDE scientific computing establishes the methodological foundation of the workflow but relies heavily on expert knowledge and manual coordination across stages. (b) Deep-learning approaches enhance individual components, including equation discovery, neural PDE solvers, surrogate modelling, and differentiable optimization, while remaining largely confined to specific tasks or workflow stages. (c) LLM-assisted systems coordinate heterogeneous scientific knowledge, executable tools, numerical solvers, and iterative feedback across the complete workflow, providing workflow-level assistance from model discovery and executable computation to downstream scientific and engineering applications.

for the **Discovery Stage**, LLMs for the **Solving Stage**, and LLMs for the **Optimization Stage**. This classification does not frame LLMs as replacements for PDE solvers, but instead asks where they contribute within the workflow that turns scientific assumptions into executable computation and task-level applications. It also enables comparison across heterogeneous studies by where LLMs intervene in the PDE workflow and what purpose they serve, rather than by final accuracy alone. Across these categories, we examine how LLMs support equation formulation and parameter identification, discretization and solver configuration, code generation and diagnostic interpretation, and PDE-constrained control, optimization, and design. Evaluation should go beyond solution accuracy to include executable reliability, numerical and physical validity, inspectability, robustness, appropriate use of solver outputs, reduction in expert burden, and overall scientific usefulness. Because current systems still depend heavily on expert-designed tools, solver interfaces, validation pipelines, and infrastructure, reliable deployment will require evidence that LLM-based assistance improves the end-to-end workflow without obscuring the underlying numerical and physical assumptions.

2 A Workflow Taxonomy of LLM Frameworks for PDEs

Consider a physical field $u(x, t)$ defined over a spatial domain Ω and a time interval $[0, T]$, whose governing equation can be written as

$$\mathcal{F}(x, t; u, \partial_t u, \nabla u, \dots; \lambda) = 0, \quad (1)$$

where $\mathcal{F}$ denotes the governing operator and λ denotes system or control parameters. The problem is completed by initial conditions $u(x, 0) = u_0(x)$ and boundary conditions $\mathcal{B}(u, \nabla u) = g(x, t)$ on $\partial\Omega$. Building on this representation, the Discovery–Solving–Application workflow can be organized according to which component of the PDE problem remains unsolved and what role the LLM plays in identifying, solving, or optimizing it.

The **Discovery Stage** concerns identifying or formulating the mathematical structure of the PDE problem. Traditionally, such structure is often derived by scientists from first principles, such as conservation laws and symmetries. In this stage, the central task for LLMs is to identify the structure of the operator $\mathcal{F}$ according to observed data and physics prior. Furthermore, once a PDE is specified, LLMs can assist with PDE-level scientific reasoning, such as deriving analytical representations or analyzing asymptotic behavior. We therefore distinguish two main Discovery-stage directions: *LLM-assisted equation discovery* and *LLM-assisted scientific reasoning*.

The **Solving Stage** aims to compute the solution $u(x, t)$ when the PDE formulation (Eq. (1)) is given. LLMs can support this process by generating solver code, configuring and orchestrating numerical software, assisting deep-learning-based PDE solvers, or interpreting diagnostic feedback such as the PDE residual, convergence behavior, stability violations, and runtime failures. We distinguish three main Solving-stage directions, including *LLM-augmented traditional solver workflows*, *LLM-based PDE solver code generation*, and *LLM-assisted deep-learning PDE solvers*.

The **Optimization Stage** treats PDE computation as part of a task-level optimization or inverse problem. Given an objective functional $\mathcal{L}$, the goal can be written abstractly as

$$\min_{\lambda} \mathcal{L}(u, \lambda) \quad \text{s.t.} \quad \mathcal{F}(x, t; u, \partial_t u, \nabla u, \dots; \lambda) = 0, \quad (2)$$

where λ represents physical parameters, control variables, or other design variables. The objective functional $\mathcal{L}$ denotes the performance objective in optimization tasks, or the discrepancy between predictions and observations in inverse problems. LLMs assist this stage by formulating objectives and constraints, proposing or refining candidate parameters or designs, coordinating solver-in-the-loop evaluations, and interpreting optimization or data-fitting feedback. We distinguish two main Application-stage directions: *LLM-assisted PDE optimization and control* and *LLM-assisted PDE-based design and model-form search*.

The three stages therefore correspond to different unresolved components of the same mathematical workflow: Discovery identifies or formulates $\mathcal{F}$ and related problem specifications, Solving computes the state $u(x, t)$, and Application optimizes λ with respect to a task-level objective $\mathcal{L}$ under the governing PDE constraint. Table 1 provides an article-level mapping of representative studies onto this workflow taxonomy, further organizing them by methodological direction, article type, and the principal function performed by the LLM.

Table 1: Representative article-level taxonomy of LLMs in PDE workflows. Each row identifies the taxonomy category, article type, and main focus of the LLM-based system or study.

Article (year)	Taxonomy category	Article type	Main focus
<i>LLMs for the Discovery Stage</i>			
LiveIdeaBench (2026) [46]	Formulation	Benchmark	Benchmarks scientific idea generation from sparse prompts.
Garnadi et al. (2025) [47]	Formulation	Case study	Assists analytical derivation for a Laplace-equation model.
Agentic Symbolic Search (2026) [48]	Formulation	Method	Searches symbolic structures for PDE solutions.
AI4Science report (2023) [49]	Formulation	Perspective / evaluation	Evaluates LLMs across scientific-discovery workflows.
LLM4ED (2024) [37]	Equation discovery	Method	Proposes and refines candidate governing equations from data.
DrSR (2025) [50]	Equation discovery	Method	Performs LLM-guided scientific symbolic regression.
LLM-SR (2025) [51]	Equation discovery	Method	Casts equation discovery as program-guided symbolic search.
Bhatnagar et al. (2025) [52]	Equation discovery	Method	Infers operator sets for PDE solution structure.
PhysPDE (2025) [38]	Equation discovery	Benchmark	Benchmarks physical-hypothesis selection for PDE discovery.
LLM and simulation as bilevel optimizers (2024) [53]	Equation discovery	Method	Couples LLM proposals with simulation-based scientific optimization.
<i>LLMs for the Solving Stage</i>			
CodePDE (2026) [39]	Solver code	Method	Generates and debugs executable PDE solver code.
Guo et al. (2025) [54]	Solver code	Method	Demonstrates LLM-assisted CAE model reduction.
PDE-SHARP (2025) [41]	Solver code	Method	Combines mathematical analysis, solver generation, and hybridization to reduce expensive solver evaluations.
PDEAgent-Bench (2026) [55]	Solver code	Benchmark	Benchmarks FEM solver-code generation by LLM agents.
Jiang et al. (2025) [56]	Solver code	Evaluation study	Evaluates LLMs on PDE and SciML coding tasks.
ALL-FEM (2026) [57]	Solver code	Agent system / fine-tuned model	Fine-tunes LLMs to generate, debug, and verify FEniCS FEM simulations.
LLM-PDEveloper (2025) [58]	Solver code	Agent system / method	Extends PDE solver libraries through agentic code development.
AutoNumerics (2026) [59]	Solver code	Agent system	Constructs PDE-agnostic numerical solvers with multiple agents.
Wang et al. (2025) [60]	Traditional solvers	Evaluation study	Evaluates LLM capabilities in computational fluid dynamics.
CFDLLMBench (2025) [61]	Traditional solvers	Benchmark	Benchmarks CFD Q&A, coding, and OpenFOAM tasks.
OpenFOAMGPT 1.0 (2025) [62]	Traditional solvers	Workflow system	Automates OpenFOAM case setup and correction.

Article (year)	Taxonomy category	Article type	Main focus
AI CFD Scientist (2026) [63]	Traditional solvers	Agent system	Runs open-ended CFD experiments and code modifications.
OpenFOAMGPT 2.0 (2026) [64]	Traditional solvers	Workflow system	Automates OpenFOAM simulation and post-processing workflows.
Foam-Agent (2025) [40]	Traditional solvers	Agent system	Generates, repairs, and executes OpenFOAM cases.
MetaOpenFOAM 1.0 (2024) [65]	Traditional solvers	Agent system	Coordinates OpenFOAM input-file generation.
MetaOpenFOAM 2.0 (2025) [66]	Traditional solvers	Agent system	Adds chain-of-thought decomposition, post-processing, and verification.
ChatCFD (2025) [67]	Traditional solvers	Agent system	Extracts, runs, and repairs CFD cases.
L2FOAM / Dong et al. (2025) [68]	Traditional solvers	Translation model / workflow	Translates natural language into executable CFD setups.
Ali-Dib (2024) [69]	Traditional solvers	Evaluation study	Tests LLMs on research-level physics simulation prompts.
MyCrunchGPT (2023) [70]	Neural solvers	Assistant / workflow system	Assists DeepONet and PINN scientific workflows.
PINNsAgent (2025) [71]	Neural solvers	Agent system	Optimizes PINN architectures and hyperparameters.
Lang-PINN (2025) [72]	Neural solvers	Agent system	Generates PINN workflows from natural-language PDE tasks.
Bao et al. (2025) [73]	Neural solvers	Evaluation study	Tests text-trained LLMs on discretised PDE dynamics.
Unisolver (2025) [74]	Neural solvers	Representation model	Conditions transformer solvers on PDE components.
UPS (2024) [75]	Neural solvers	Representation / foundation model	Adapts pretrained language models to PDE solving.
FLUID-LLM (2024) [76]	Neural solvers	Representation model	Predicts spatiotemporal fluid dynamics with LLM-style models.
Text2PDE (2025) [77]	Neural solvers	Representation / generative model	Generates text-conditioned PDE rollouts.
FLUID-GPT (2023) [78]	Neural solvers	Representation model	Models particle trajectories and erosion fields.
<i>LLMs for the Optimization Stage</i>			
AeroAgent (2026) [79]	Design and model-form search	Agent system	Optimizes aerodynamic vehicle shapes with simulation guidance.
ShapeBench (2026) [80]	Design and model-form search	Benchmark	Benchmarks aerodynamic shape-optimization algorithms.
OptMetaOpenFOAM (2025) [81]	Optimization and control	Workflow system	Orchestrates CFD sensitivity analysis and parameter optimization.
PDE-Controller (2025) [42]	Optimization and control	Control framework	Formalises PDE-control goals and synthesises control programs.
Sun et al. (2026) [82]	Optimization and control	Agent system / control discovery	Discovers interpretable fluid-control policies through LLM-generated controller code and simulation feedback.
Zhang et al. (2025) [43]	Design and model-form search	Design-optimization method	Guides parametric shape optimization with simulation feedback.

Article (year)	Taxonomy category	Article type	Main focus
Yang et al. (2025) [83]	Design and model-form search	Method	Develops near-wall turbulence closure models.
Lee et al. (2025) [84]	Design and model-form search	Perspective	Frames knowledge-guided inverse design for manufacturing.
Kumar and Karniadakis (2025) [85]	Design and model-form search	Multi-agent framework	Coordinates knowledge-guided agents for autonomous aerodynamic engineering design.

3 LLMs for the Discovery Stage

Identifying the mathematical form of a PDE and investigating its underlying properties constitute two fundamental aspects of the PDE workflow. The former concerns discovering governing equations that effectively describe physical systems, while the latter involves formulating mathematically well-defined problems and reasoning about analytical solutions, structural properties, and qualitative behavior. PDE discovery remains a longstanding scientific challenge, although advances in deep learning have enabled increasingly complex libraries of candidate operators to be searched and screened in a partially automated manner to identify equations that are more consistent with observational data [4, 15, 16]. LLMs introduce a complementary capability to this process: the scientific knowledge and physical reasoning capabilities encoded in these models can help generate, prioritize, and refine plausible hypotheses, thereby partially assuming the role traditionally played by expert intuition in scientific discovery. Equally important is reasoning from a given PDE to derive its analytical solutions, structural properties, and qualitative behavior, which forms a fundamental part of theoretical PDE research. The reasoning capabilities of LLMs provide new opportunities to assist such mathematical analysis and scientific inference. We therefore distinguish two main directions: *LLM-assisted equation discovery* and *LLM-assisted problem formulation and scientific reasoning*. Fig. 2 illustrates representative workflows for both directions.

3.1 LLM-assisted equation discovery

Equation discovery aims to recover the governing PDE of a physical system from observational data. Traditionally, this process relies heavily on expert physical intuition to construct candidate equations or operator libraries and to identify the mathematical expression that best explains the observations. As illustrated in Fig. 2a, the scientific and physical priors encoded in LLMs can participate directly in this process by constructing, screening, or constraining the candidate search space, and can even be used to propose complete candidate equations directly.

For the first path, in which PDEs are assembled by searching over combinations of candidate operators, the motivation for introducing LLMs arises from a central limitation of conventional equation-discovery methods: they often depend either on manually specified operator libraries or on largely unconstrained symbolic search. An overly restrictive candidate library may exclude the true governing terms, whereas an excessively broad one can lead to combinatorial explosion and a proliferation of spurious structures. Existing approaches therefore intervene at different points along this pipeline to address complementary bottlenecks in conventional equation discovery. Before symbolic search, language models can infer problem-specific operators, intermediate physical properties, or structural constraints that determine what the downstream regression engine is allowed to explore. This can reduce dependence on manually specified candidate libraries and alleviate the trade-off between missing the true governing structure and searching an unnecessarily vast hypothesis space. KeplerAgent, for example, infers properties such as symmetries and plausible functional

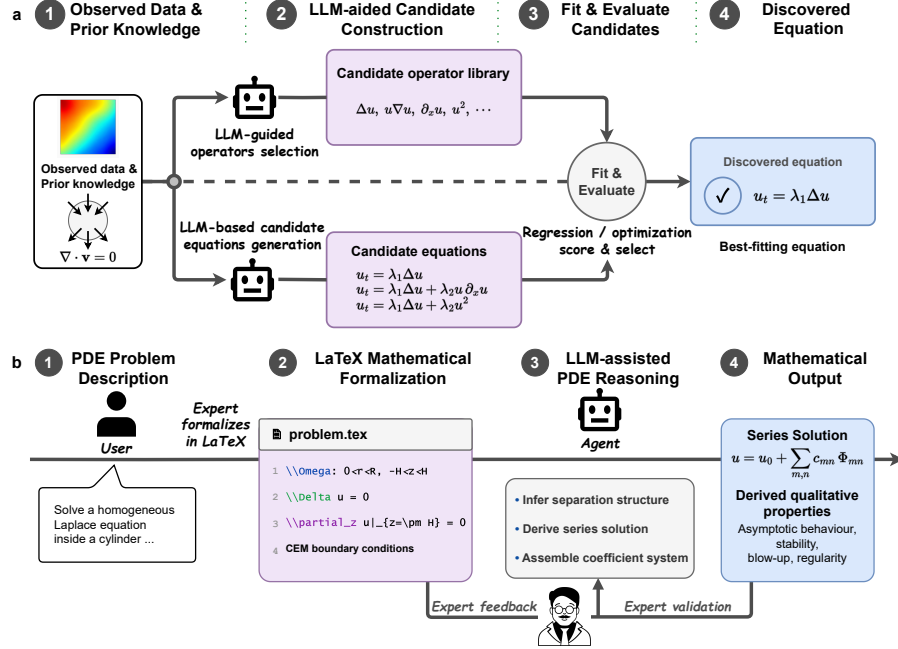


Figure 2: Representative LLM-assisted workflows in the Discovery Stage. (a) LLM-assisted equation discovery and system identification. Observations and prior scientific knowledge guide either the construction of a candidate operator library or the direct generation of candidate equations. Candidate structures are then fitted, evaluated, and selected through regression, optimization, simulation, or other validation procedures before a plausible governing equation is retained. (b) LLM-assisted problem formulation and scientific reasoning. An initially informal PDE problem is first formalized as a mathematical specification, after which an LLM supports structured PDE reasoning, symbolic derivation, or qualitative analysis under expert feedback and validation.

structures and uses them to configure symbolic-regression engines, including their candidate libraries and structural constraints [86]. By injecting such intermediate physical knowledge before search, it narrows the hypothesis space toward structures that are more consistent with the underlying scientific problem. Similarly, Bhatnagar et al. predict compact operator sets for symbolic PDE solution representations, thereby restricting the subsequent search for analytical approximations and reducing the burden of exploring unnecessarily large operator combinations [52]. Language models also intervene after a candidate expression has been recovered, where the main challenge shifts from search-space construction to scientific interpretation. PhysPDE evaluates such a pipeline by mapping candidate PDE expressions to plausible physical hypotheses, helping bridge the gap between mathematically valid expressions and scientifically meaningful explanations [38]. Although these methods intervene at different stages, they share the same broader principle: the LLM does not determine the final mathematical expression through numerical fitting itself, but instead uses learned scientific knowledge to make the discovery process more targeted, physically informed, and interpretable.

For the second path, in which complete candidate equations are generated directly, the motivation for introducing LLMs lies in the enormous discrete structural space that conventional symbolic regression must search over, including variables, operators, functions, and their possible compositions. Without informative structural priors, random search, evolutionary search, and other strategies need to evaluate large numbers of invalid, redundant, or physically implausible candidates, while making limited use of established mathematical patterns, scientific knowledge, or historical evaluation results to decide what should be explored next. LLMs are therefore introduced

into an iterative generation–evaluation–revision loop, where their encoded mathematical and scientific knowledge can be combined with numerical fitting results, historical candidate performance, or explicit data-aware reasoning to propose more promising equation structures and continuously redirect the subsequent search. Recent methods follow this pattern by generating candidate equations or equation skeletons, numerically fitting or evaluating them, and then revising them using learned mathematical patterns, parameter-fitting results, historical candidate performance, or explicit data-aware reasoning and reflection [37, 50, 51, 87]. The role of the LLM is therefore not merely to produce an initial expression, but to determine which mathematical structures should be explored next and how promising candidates should be revised in response to executable feedback.

Across both paths, the distinctive contribution of the LLM lies in transforming scientific knowledge from an implicit expert resource into an active search prior that guides which mathematical structures are considered, how they are revised, and which hypotheses are pursued next. Although candidate equations still require numerical fitting, physical validation, and expert judgment [3, 16, 38], LLMs offer a new way to integrate scientific knowledge directly into the search process, making equation discovery more targeted, physically informed, and adaptive.

3.2 LLM-assisted problem formulation and scientific reasoning

Beyond discovering unknown governing equations, another fundamental challenge in the Discovery Stage is to formulate mathematically explicit PDE problems and investigate the knowledge encoded in their mathematical structure. Existing studies mainly cover two related tasks: transforming incomplete scientific intent into well-defined PDE problems, and reasoning from a given PDE to derive analytical solutions, identify structural properties. The motivation for introducing LLMs is particularly clear in this context. Both formulation and mathematical analysis traditionally require lengthy, multi-step, and expert-intensive reasoning, in which domain specialists must combine physical knowledge, mathematical intuition, problem constraints, and intermediate derivations to decide how a problem should be posed or which reasoning direction should be pursued next. By drawing on learned scientific priors and mathematical reasoning capabilities, LLMs can partially automate this traditionally manual cognitive process and make expert knowledge an active component of the formulation and reasoning workflow.

As illustrated in Fig. 2b, one direction uses LLMs to assist the derivation of analytical representations and structural properties from already specified PDE problems. Garnadi’s case study, for example, uses an LLM to assist the analytical derivation of a homogeneous Laplace-equation model with the complete electrode model, including reasoning over cylindrical coordinates, separation-of-variables structures, and boundary conditions [47]. Here, the value of the LLM lies in helping automate a chain of mathematical decisions and intermediate derivations that would otherwise require substantial expert effort. Agentic Symbolic Search goes further by allowing language-model agents to translate PDE theory, problem constraints, and accumulated search experience into differentiable symbolic programs, which are then refined through evolutionary search and continuous parameter optimization to recover explicit analytical forms or interpretable approximations of PDE solutions [48]. The distinctive role of the LLM is therefore not simply to manipulate symbols, but to inject mathematical priors into the search itself: it determines which structures are plausible, how previous search experience should influence subsequent proposals, and which analytical forms deserve further exploration.

A related direction concerns problem formulation from incomplete or heterogeneous scientific information. Multimodal PDE foundation models illustrate an important prerequisite for such formulation by integrating numerical inputs, such as equation parameters and initial conditions, with textual descriptions of physical processes or system dynamics when symbolic representations

are incomplete or unavailable [88]. The relevance of LLMs here stems from their ability to operate across heterogeneous representations that conventional numerical solvers or symbolic tools do not naturally unify, providing a possible interface between informal scientific descriptions, numerical states, and explicit mathematical structure.

More broadly, scientific-AI benchmarks and perspective studies reveal another characteristic that makes LLMs particularly relevant to scientific reasoning: they can draw on broad parametric scientific knowledge to expand sparse scientific cues into candidate ideas or hypotheses. LiveIdeaBench, for example, evaluates whether LLMs can generate scientifically meaningful ideas from minimal single-keyword prompts, thereby probing their ability to rely primarily on internal scientific knowledge rather than detailed contextual scaffolding [46]. Broader AI-for-science studies likewise demonstrate the capacity of LLMs to integrate scientific knowledge and reason across domains including biology, chemistry, materials science, and PDEs [49]. These studies do not by themselves establish reliable PDE formulation or mathematical discovery, but they highlight a capability central to both: using prior scientific knowledge to propose what assumptions, structures, hypotheses, or reasoning directions should be considered when the problem is underspecified. Across these tasks, the distinctive contribution of the LLM lies in turning expert reasoning itself from a manual bottleneck into a partially automatable component of the PDE workflow.

4 LLMs for the Solving Stage

Once a PDE problem is sufficiently specified, the central challenge shifts from determining what equation should be studied to transforming that equation into a reliable computational process. This transformation is rarely a single act of numerical solution. It requires translating mathematical specifications into executable representations, selecting numerical methods and software configurations, generating or modifying code, diagnosing failures from logs and numerical feedback, and, in learned solvers, choosing architectures, losses, conditioning variables, and training procedures. These decisions are traditionally distributed across mathematical reasoning, numerical expertise, software documentation, source code, and iterative experimentation, making PDE solving highly dependent on expert intervention.

The motivation for introducing LLMs lies precisely in their ability to operate across these heterogeneous representations. Unlike fixed automation scripts or task-specific predictive models, LLMs can interpret natural-language and symbolic specifications, retrieve and synthesize technical knowledge, generate executable code, interact with external tools, and revise previous decisions in response to runtime or numerical feedback. Their role is therefore not to replace numerical solvers or numerical analysis, but to reduce the representational and procedural gap between a specified PDE and its executable solution workflow.

As summarized in Fig. 3, the three directions differ in the computational object around which the LLM operates: an established numerical-solver workflow, a numerical programme generated from a PDE specification, or a learned PDE solution model. We accordingly distinguish *LLM-augmented traditional solver workflows*, *LLM-based PDE solver code generation*, and *LLM-assisted deep-learning PDE solvers*. Across all three directions, the central question is therefore not whether an LLM can replace a PDE solver, but whether its language understanding, scientific knowledge, code generation, tool use, and feedback-driven reasoning can resolve workflow bottlenecks that are difficult to address through fixed rules or narrowly specialized models alone.

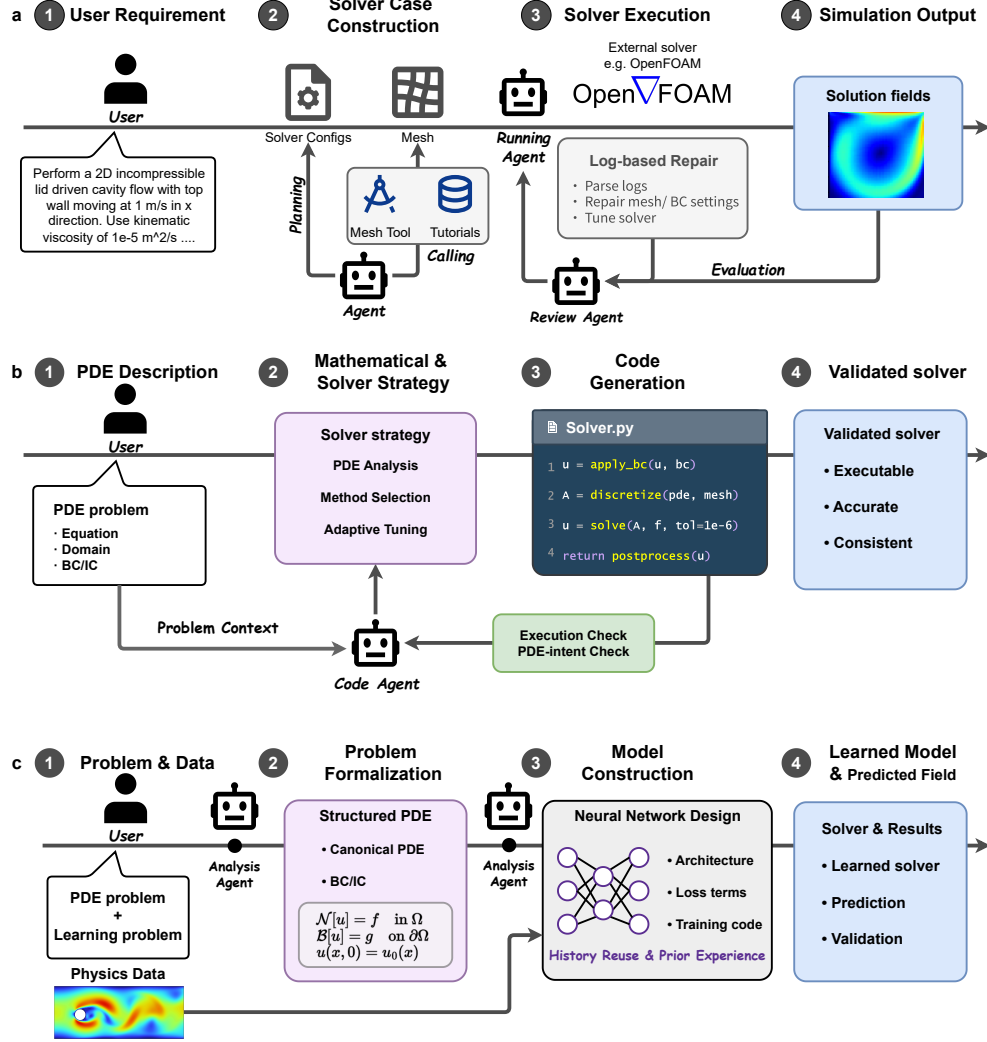


Figure 3: LLMs for PDE solving-stage workflows. a, In traditional solver workflows, LLM agents translate PDE specifications into solver configurations, execute external tools, and revise failed cases using logs and diagnostics. b, In solver-code generation, LLMs generate, test, and debug numerical programmes from mathematical or natural-language PDE descriptions. c, In deep-learning PDE solvers, LLMs assist architecture design, loss construction, training workflows, or language-conditioned PDE representations.

4.1 LLM-augmented traditional solver workflows

Traditional PDE and CFD solvers are already powerful numerical engines; the main difficulty lies in turning a mathematical problem into a reliable executable workflow. A complete simulation requires coordinating multiple components from geometry to numerical schemes. The necessary knowledge is often distributed across scientific descriptions, software documentation, tutorial cases, source files, and runtime logs, while failures may arise from context-dependent inconsistencies spanning several components of the workflow. Fixed scripts can automate predefined procedures, but they are difficult to adapt when problem specifications change, solver choices depend on physical context, or unexpected failures require reasoning across heterogeneous information. The motivation for introducing LLMs therefore lies not in replacing established numerical solvers, but in using natural-language understanding, domain knowledge integration, tool interaction, and feedback-driven correction to coordinate solver-centred workflows. This division of labour is illustrated in Fig. 3a: the

established solver remains responsible for numerical computation, while the LLM interprets user requirements, coordinates case construction and execution, and uses diagnostic feedback to revise the workflow.

A recurring role of LLMs in this setting is to bridge high-level scientific specifications and the detailed configurations required by numerical software. OpenFOAM-based systems provide the clearest examples. OpenFOAMGPT combines retrieval-augmented generation with solver execution and iterative correction to translate user requirements into OpenFOAM cases and revise failed configurations [62]. MetaOpenFOAM similarly decomposes natural-language CFD tasks into executable subtasks spanning preprocessing, solver setup, and simulation, while grounding agents in OpenFOAM tutorials and iteratively correcting errors [65]. NL2FOAM follows a complementary route by adapting language models to natural-language-OpenFOAM configuration pairs and embedding generation within a workflow of input checking, execution, and correction [68]. These systems ground solver-specific knowledge and organize the workflow in different ways, including retrieval, multi-agent task decomposition, and domain adaptation. Despite these differences, they address the same underlying bottleneck: a scientific problem description does not map directly to an executable CFD case. Reliable translation requires simultaneous reasoning over physical intent, numerical choices, software syntax, and dependencies among solver files and configurations.

Beyond specification-to-case translation, LLMs are increasingly used to coordinate longer and more heterogeneous sequences of solver operations. MetaOpenFOAM 2.0 extends multi-agent decomposition with iterative verification and post-processing, while OpenFOAMGPT 2.0 distributes preprocessing, simulation, and post-processing across specialized agents [64, 66]. Foam-Agent further spans mesh generation, case setup, high-performance-computing scripts, execution, debugging, and visualization, combining hierarchical retrieval with dependency-aware file generation and modular tool use [40]. The distinction is not the mere presence of execution feedback or error correction, which already appear in earlier systems such as OpenFOAMGPT and MetaOpenFOAM. More recent systems instead extend LLM assistance from local setup and repair to longer solver workflows, where intermediate files, logs, diagnostics, and post-processing outputs guide subsequent choices about configuration, execution, revision, and validation. This need for cross-step reasoning becomes especially apparent when execution fails. The LLM must reason across physical intent, software syntax, configuration dependencies, and runtime evidence to decide what should be revised next. ChatCFD emphasizes this diagnostic role through structured domain knowledge, explicit error localization, coordinated cross-file correction, and reflection over persistent failures [67]. AI CFD Scientist further extends this solver-centred paradigm toward more open-ended computational investigation by connecting literature-grounded task formulation, parameter studies, local model modification, simulation execution, and multimodal inspection of predicted flow fields [63].

A CFD case may run successfully while still encoding inappropriate boundary conditions, physical models, numerical schemes, or mesh choices. CFDLLMBench therefore evaluates LLMs across CFD knowledge, numerical and physical reasoning, code implementation, and OpenFOAM workflow execution, with criteria including executability, solution accuracy, and convergence behaviour [61]. Numerical and physical validation remain essential, but the distinctive value of LLMs lies in making a fragmented solver workflow adaptive: they connect scientific specifications, domain knowledge, solver files, external tools, and diagnostic feedback to determine which action should be taken or revised next.

4.2 LLM-based PDE solver code generation

A second subcategory treats PDE solving as a specification-to-program problem. Translating a PDE into executable code is not a direct transcription task, because the mathematical specifica-

tion rarely determines a unique discretization, time-integration scheme, boundary treatment, solver configuration, or software implementation. Template-based generators can automate predefined combinations, and generic code models can reproduce familiar programming patterns, but they struggle when the implementation requires context-dependent numerical choices or when execution failures must be traced back to mathematical and physical requirements. LLMs are particularly relevant because they can reason jointly over equations, numerical methods, software interfaces, generated code, and test results, allowing implementation decisions to be revised as new evidence becomes available. Figure 3b illustrates this coupled specification–implementation–validation loop, in which mathematical analysis, method selection, code generation, execution checks, and PDE-specific numerical tests jointly determine whether the resulting programme can be accepted as a solver.

CodePDE is representative of this direction. It frames PDE solving as LLM-driven solver generation and studies inference-time mechanisms such as reasoning, debugging, self-refinement, and test-time scaling [39]. Its workflow highlights a broader pattern: solver generation is not a single code-completion step, but a loop that connects task specification, implementation, execution, evaluation, and refinement. LLM-empowered CAE automates the mathematical derivation, code adaptation, and verification required to construct reduced-order solvers for new parametric PDEs, thereby enabling faster repeated evaluations within downstream design and optimization workflows [54]. ALL-FEM constructs a large collection of verified FEniCS programmes and uses it to adapt language models to variational formulations, finite-element software conventions, code generation, debugging, and result verification [57]. This shows that reliability can be improved not only through inference-time refinement, but also by training models on validated numerical implementations. PDEAgent-Bench further formalizes this evaluation setting by introducing a multi-metric, multi-library benchmark for PDE-to-solver code generation, where generated solvers must pass staged checks for executability, numerical accuracy, and computational efficiency across finite-element libraries [55]. This benchmark perspective is useful because generic code benchmarks rarely capture the numerical requirements of PDE software.

Other work targets solver libraries and broader numerical-code development. Automated PDE code-development frameworks use LLMs to translate mathematical and algorithmic descriptions into source-code modifications, such as adding new solvers, boundary conditions, or equation terms to existing PDE libraries [58]. AutoNumerics extends this idea toward a PDE-agnostic multi-agent pipeline that designs, implements, debugs, and verifies numerical solvers from natural-language problem descriptions [59]. PDE-SHARP explores another axis of solver generation: it uses mathematical analysis, solver generation, and synthesis or hybridization passes to reduce the number of expensive solver evaluations needed during test-time search [41]. These systems suggest that LLMs can participate not only in writing code, but also in selecting numerical ideas, organizing refinement, and trading off LLM inference against costly scientific computation.

The evaluation burden in this subcategory is especially high. A generated solver may be syntactically valid and even produce plausible plots while still using an unstable discretization, imposing the wrong boundary condition, violating conservation, or converging to the wrong solution. Studies comparing LLMs on scientific computing and CFD tasks show that model performance depends strongly on reasoning ability, task design, and the requirement to make non-trivial numerical decisions [56, 60]. Benchmarks that enforce numerical accuracy and efficiency therefore provide a stronger test than simple executability checks [55, 61]. For review purposes, this distinction is central: PDE solver generation is not merely software engineering, because the generated code is meaningful only if it respects the mathematical and physical structure of the PDE.

The main promise of LLM-based solver code generation is that it can reduce the distance between mathematical specifications and executable numerical experiments. However, generated

programmes remain candidate implementations until they have passed unit tests, manufactured-solution tests, residual and convergence checks, conservation diagnostics, and comparisons with trusted solvers. The distinctive value of the LLM lies in coupling mathematical reasoning, implementation, execution, and refinement within a single adaptive loop, rather than in treating code generation alone as evidence of numerical correctness.

4.3 LLM-assisted deep-learning PDE solvers

A third subcategory uses LLMs primarily to assist the construction and operation of deep-learning PDE solvers. Building a usable PINN, DeepONet, or related learned solver requires coordinated decisions about PDE representation, physics-informed loss terms, sampling strategies, network architecture, optimization schedules, implementation, and training diagnosis. These choices are strongly interdependent, and the appropriate revision often depends on symbolic problem information, prior experience, generated code, and observed training behaviour. Fixed AutoML procedures can optimize predefined hyperparameters, but they are less suited to open-ended revisions that require interpreting PDE semantics and numerical feedback together. LLMs are therefore valuable because they can combine scientific knowledge, symbolic reasoning, code generation, memory, and execution feedback within a single adaptive workflow. As illustrated in Fig. 3c, the LLM helps organize the PDE problem and available data, construct the learned solver, and revise the model or training procedure according to validation results.

PINN-oriented systems illustrate the automation side of this subcategory. An early example is MyCrunchGPT, which uses a conversational language model to coordinate backend modules for PINNs, DeepONet, geometry processing, visualization, and numerical assessment [70]. PINNsAgent uses LLMs to automate PDE surrogation with physics-informed neural networks, combining structured knowledge replay from solved PDEs with a memory-based reasoning strategy for exploring PINN architectures and configurations [71]. Lang-PINN moves closer to an end-to-end workflow: it translates natural-language task descriptions into symbolic PDEs, selects PINN architectures, generates implementations, and uses execution feedback for iterative refinement [72]. Across these systems, the LLM’s role extends beyond hyperparameter selection: it uses symbolic problem information, prior experience, generated implementations, and training feedback to determine which part of the learned-solver workflow should be revised next.

A related but less workflow-oriented direction incorporates language-model-derived architectures or conditioning representations into learned PDE solvers. UPS, Unisolver, FLUID-LLM, Text2PDE, text-trained PDE models, and FLUID-GPT investigate whether pretrained language-model components or textual and symbolic conditioning can provide transferable representations across heterogeneous PDE families [73–78]. In these studies, the LLM contribution lies mainly in representation transfer or cross-modal conditioning rather than in reasoning over the solver workflow itself.

Evaluating learned PDE solvers differs from standard code generation: while they can achieve low error on benchmark rollouts, they often fail under altered physical configurations, coefficients, or time horizons. Consequently, assessment must extend beyond test-set accuracy to verify whether these models strictly respect physical constraints, extrapolate across regimes, maintain rollout stability, and remain scientifically interpretable.

Across the three subcategories, LLMs are relevant for a common reason: the next computational action cannot always be specified in advance, but must be inferred from a changing combination of mathematical descriptions, domain knowledge, software conventions, source code, training histories, and numerical feedback. Established numerical or learned solvers provide the computational engine, while the LLM connects these forms of information and determines how the surrounding workflow should be configured, constructed, diagnosed, or revised. The credibility of these decisions still

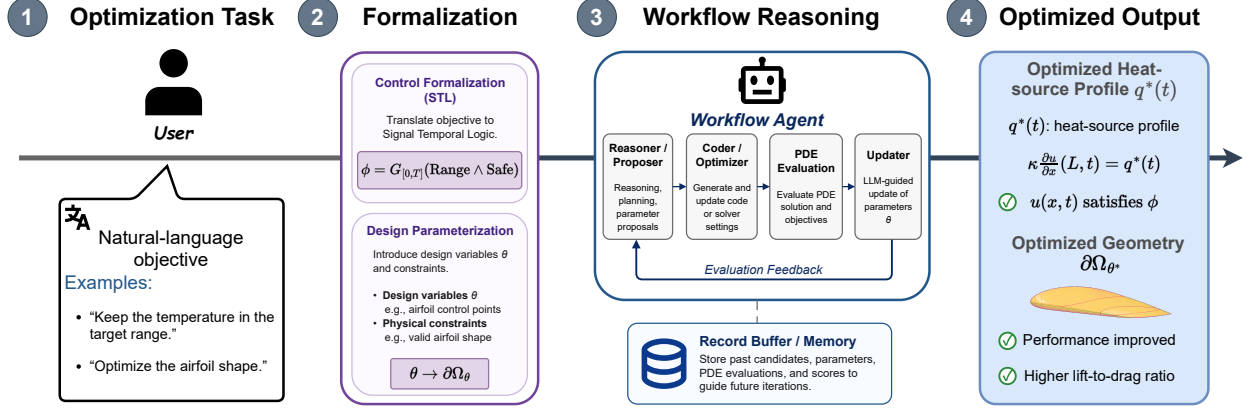


Figure 4: LLMs for PDE optimization-stage workflows. A natural-language task is first translated into an explicit objective, constraint set, and decision-space specification. A workflow agent then coordinates computational tools and an iterative search loop in which candidate parameters, control programs, geometries, or model structures are proposed, evaluated through PDE solvers or surrogate models, compared with previous candidates, and revised using physical and task-level feedback. The resulting output can be an optimized control input, such as $q^*(t)$, or an optimized design, such as $\partial\Omega_{\lambda^*}$.

depends on executable validation, numerical diagnostics, and physical interpretation. The strongest systems are therefore not those that hide the solver, but those that make the computation traceable by exposing generated files, code, logs, residuals, convergence histories, validation tests, and modelling assumptions. This provides the bridge to the Optimization Stage, where PDE computation becomes part of a larger loop for optimization, control, or design.

5 LLMs for the Optimization Stage

The **Optimization Stage** concerns PDE-based decision-making after the PDE problem has been identified or formulated and a usable computational procedure has been established. The task is no longer only to compute the state $u(x, t)$, but to use repeated PDE evaluations to select physical parameters, control inputs, geometries, material distributions, or model structures. Following the notation introduced in Eq. (1), this task can be written abstractly as

$$\min_{\lambda \in \Lambda} \mathcal{L}(u, \lambda) \quad \text{subject to} \quad \mathcal{F}(x, t; u, \partial_t u, \nabla u, \dots; \lambda) = 0, \quad \mathcal{C}(u, \lambda) \leq 0, \quad (3)$$

where λ denotes the decision variables, Λ is the admissible decision space, $\mathcal{L}$ is the task-level objective, and $\mathcal{C}$ represents physical, operational, or design constraints.

In practice, domain experts must interpret the scientific goal, define the objective and constraints, choose a parameterization, connect optimizers with PDE solvers or surrogate models, evaluate candidate decisions, and revise the process according to simulation results. Established optimizers search efficiently once the variables, objective, constraints, and update rules have been defined, while PDE solvers provide quantitative evaluations of individual candidates. However, these tools do not by themselves interpret informal scientific requirements, compose heterogeneous software components, or translate unstructured physical and engineering knowledge into workflow and candidate-proposal decisions. Fixed workflow systems automate predefined pipelines, but they struggle when requirements change, failures span several components, or the next operation depends jointly on mathematical specifications, software states, and diagnostic feedback. LLMs address these gaps by reasoning across natural-language intent, symbolic problem descriptions, scientific knowledge, code, computational tools, and optimization history. As illustrated in Fig. 4, LLMs integrate

heterogeneous components into an executable optimization workflow and, within this process, use scientific and engineering priors to enhance candidate generation.

Workflow integration addresses the extensive coordination required to turn a scientific request into an executable optimization loop. PDE-Controller translates informal descriptions of desired system behavior into formal specifications, including signal temporal logic (STL) constraints, and then supports reasoning, planning, and program synthesis for PDE-governed control problems [42]. It connects linguistic intent with executable control evaluation, while the governing PDE and external evaluator determine whether a candidate controller satisfies the specification. OptMetaOpenFOAM connects a natural-language interface, the MetaOpenFOAM environment, and external sensitivity-analysis and optimization libraries [81]. A user request can therefore initiate case construction, repeated simulations, post-processing, sensitivity analysis, and parameter updates. Across these systems, the LLM does not replace the optimizer or PDE solver; it reduces the expert effort required to formalize the task, compose computational components, interpret intermediate results, and maintain an executable optimization loop when the required sequence of operations cannot be completely specified in advance.

Within such an integrated workflow, LLMs can further use scientific and engineering priors to guide candidate generation. Conventional optimization methods already provide effective mechanisms for updating parameters and selecting candidates from objective values, gradients, or pre-defined search rules. Their performance, however, depends strongly on how the decision space, parameterization, constraints, and proposal mechanisms are constructed. PDE optimization often involves additional knowledge, including physical mechanisms, engineering preferences, structural relations, and experience from previous simulations, that is difficult to encode completely in a numerical objective or fixed update rule. LLMs provide a complementary proposal mechanism by using this knowledge to identify physically plausible candidates, preserve important design requirements, prioritize promising regions or structures, and avoid evaluations that conflict with known physical or engineering constraints.

The clearest examples place the LLM directly inside the proposal–evaluation loop. Zhang et al.’s LLM-PSO uses previously evaluated shape parameters and objective values to generate new geometric candidates [43]. Sun et al. combine LLM-generated controller code with simulation feedback to search for interpretable fluid-control policies [82]. LLM-driven turbulence-model development extends the same principle to model-form search, using physical reasoning together with a priori and a posteriori evaluation to propose and revise interpretable closure structures [83]. These systems do not replace quantitative optimization or simulation-based evaluation. Instead, they use the LLM to introduce scientific context into the proposal process, making expensive PDE evaluations more targeted and better informed by prior knowledge.

Related studies connect this search role to broader engineering-design workflows. AeroAgent coordinates domain knowledge, aerodynamic analysis, candidate development, and simulation-guided revision, but its contribution extends beyond direct search guidance to the organization of the wider design process [79]. Knowledge-guided multi-agent frameworks similarly combine specialized agents, simulation tools, and iterative design operations [85]. Lee et al. provide a perspective rather than an implemented optimization system, outlining how physical laws, domain knowledge, simulation, data, and human judgement could be integrated within future knowledge-guided inverse-design frameworks [84]. Together, these studies indicate that LLMs are most valuable when optimization benefits from knowledge that cannot be represented adequately by objective values and numerical variables alone.

6 Benchmarking and Evaluation across the PDE Workflow

Benchmarking LLM-based PDE workflows is challenging because the field spans diverse tasks across the Discovery, Solving, and Optimization stages, each requiring different forms of evidence. A dedicated evaluation perspective is therefore needed to clarify what constitutes success in each direction, distinguish genuine workflow improvements from superficial task completion, and identify gaps in current benchmarks. Table 2 summarizes representative evaluation resources and metrics for the main directions reviewed above.

Table 2: Representative benchmarks, datasets, fine-tuning corpora, and evaluation metrics for LLM-based PDE workflows across the Discovery, Solving, and Optimization stages. Because the workflow directions differ substantially in their inputs, outputs, and validation requirements, the listed metrics should be interpreted as complementary rather than interchangeable.

Workflow direction	Representative Benchmarks / Datasets / Corpora	Main Metrics
LLM-assisted equation discovery	Physical hypothesis selection datasets [38]	Expression Recovery Term-Selection Accuracy Physical Plausibility
LLM-assisted problem formulation and scientific reasoning	LiveIdeaBench [46] Analytical PDE reasoning tasks [47]	Formulation Completeness Mathematical Correctness Reasoning Traceability
LLM-augmented traditional solver workflows	OpenFOAM tutorials [89] CFDLLMBench [61]	Task Success Recovery Rate Human Interventions Execution Cost
LLM-based PDE solver code generation	PDEBench [90] CFDCodeBench [61] ALL-FEM verified FEniCS corpus [57] AutoNumerics [59]	Executable Rate Code-level Success Numerical Accuracy Compilation Success Runtime
LLM-assisted deep-learning PDE solvers	PINNacle [91]	Prediction Accuracy Constraint Satisfaction Cross-Context Generalization
LLM-assisted PDE optimization	Parameter-optimization workflows [81] PDE-control formalization tasks [42] Parametric shape optimization [43] ShapeBench high-fidelity CFD/FEA [80]	Task Completion Formalization Accuracy Objective or Utility Gain Budgeted Convergence Feasibility / Fidelity Gap

Current evaluation practice remains uneven. Benchmarks are relatively more developed for Solving-stage tasks, where success can be assessed through code executability, workflow completion, failure recovery, numerical accuracy, or convergence. Evaluation remains less mature in parts

of the Discovery Stage, where the central questions concern mathematical completeness, reasoning transparency, physical plausibility, and whether a proposed formulation is usable downstream. Optimization-stage evaluation is broader again: objective improvement must be interpreted alongside sample efficiency, constraint satisfaction, physical validity, robustness, and the amount of expert intervention still required.

A more fundamental limitation is that current benchmarks often emphasize proxy outcomes rather than the central question of this field: whether expert burden has been reduced in a scientifically meaningful way. This is especially apparent for solver-orchestration systems, where high success rates on curated cases do not necessarily demonstrate reliability on unfamiliar, less templated workflows. Evidence is still needed that LLM-based systems remain effective when geometries, boundary conditions, software interactions, numerical failures, and physical regimes differ substantially from the examples used during development.

Future benchmarks should therefore move beyond static task completion and evaluate burden reduction, scientific validity, and long-horizon workflow behaviour. They should measure how much expert intervention remains necessary, whether systems can identify scientifically meaningful failure modes rather than merely react to predefined errors, and whether they remain reliable when intermediate decisions and solver feedback alter the subsequent workflow. Benchmark design is therefore not peripheral to this field; it will strongly shape which forms of LLM assistance are regarded as credible progress.

7 Challenges and Outlook

The central conclusion of this Review is that LLMs contribute most credibly to PDE research at the workflow level. PDE-centred computation distributes expert work across problem formulation, model identification, solver construction, numerical diagnosis, result interpretation, and downstream decision-making. LLMs can reduce this burden by connecting natural-language intent, mathematical representations, scientific knowledge, code, computational tools, and feedback across these activities. Existing studies demonstrate parts of this role in equation discovery and scientific reasoning [37, 38, 48]. Other systems extend it to solver configuration, numerical-code generation, and learned-solver construction [39, 62, 71]. Optimization-stage systems further formalize decision problems, coordinate repeated PDE evaluations, and use scientific or engineering knowledge to guide candidate generation [42, 43, 81]. The key question is therefore whether LLMs can reduce manual coordination across the complete PDE workflow without weakening numerical reliability or scientific credibility.

The first challenge is that current systems still operate within structures largely designed by domain experts. In Discovery, humans define admissible operators, physical priors, search spaces, fitting procedures, and validation criteria [37, 38, 52]. In Solving, agentic systems depend on solver documentation, tutorial cases, predefined software interfaces, available tools, and externally supplied diagnostics [40, 62, 65]. In Optimization, decision variables, parameterizations, objectives, constraints, and high-fidelity evaluators are normally established before the LLM begins to act [42, 43, 81]. These dependencies are also the main channels through which scientific judgement enters the workflow. Current demonstrations therefore show primarily that LLMs can make expert-designed procedures more executable and accessible. They do not yet establish that LLMs can reliably select scientifically valuable questions, modelling assumptions, numerical strategies, and validation standards without substantial human framing and verification.

A second challenge is maintaining coherent behaviour across long computational workflows. Real PDE studies involve equations, meshes, configuration files, generated code, solver logs, post-processing routines, diagnostic plots, and records of previous failures. A decision made during

problem formulation may later affect solver configuration, numerical stability, interpretation, and optimization. Current systems already use retrieval, memory, task decomposition, and dependency-aware generation to maintain parts of this state [40, 64, 71]. Multi-agent frameworks also connect preprocessing, execution, debugging, and verification across several tools [59, 66]. Reliability nevertheless remains difficult when specifications change, failures span several components, or intermediate results require the workflow itself to be reorganized. Future systems must preserve assumptions, decisions, code changes, failures, and validation evidence over long horizons, rather than treating each generation or tool call as an isolated step.

A deeper challenge is the transition from reacting to prescribed feedback towards conducting scientific critique. Most current systems respond to signals whose meaning has already been defined by humans, including parser errors, failed tests, large residuals, non-converged simulations, and disagreement with reference solutions [39, 59, 67]. Repairing these failures is useful, but it is not equivalent to scientific judgement. Experts decide which conserved quantities should be inspected, which nondimensional groups matter, which parameter regimes are most revealing, and when apparent agreement is insufficient evidence. Verification and validation principles have long emphasized that successful execution or agreement on a limited set of cases does not establish model credibility [3]. More capable LLM-based systems should therefore help identify plausible failure modes, select informative diagnostics, design stress tests, compare competing explanations, and recognize when the available evidence cannot support a modelling or design conclusion.

These challenges point to three enabling directions. First, reasoning-time computation and iterative refinement should allow systems to compare alternative formulations, numerical strategies, and implementations, rather than improving only a single prescribed solution path [39, 41]. Second, provenance-aware memory and retrieval should preserve modelling assumptions, code modifications, solver states, failed attempts, and validation evidence across extended workflows. Third, executable and multimodal tool use should support active scientific verification by connecting symbolic algebra, mesh generation, numerical solvers, finite-element libraries, numerical fields, diagnostic plots, and error messages [63, 88]. The key advance will be the ability to revise the scientific plan itself when accumulated evidence challenges earlier assumptions, rather than merely calling additional tools or correcting predefined errors.

These capabilities imply different priorities across the three stages. In the Discovery Stage, progress requires moving from unconstrained equation proposal towards physically grounded hypothesis construction. LLMs should help identify relevant variables and mechanisms, compare alternative governing structures, and design simulations or experiments that distinguish among competing explanations [37, 38, 48]. In the Solving Stage, the priority is robust executable workflows that combine solver configuration, numerical-code generation, mesh and boundary-condition handling, testing, diagnosis, and validation across unfamiliar problems [39, 57, 59]. In the Optimization Stage, LLMs can connect informal objectives with explicit constraints, coordinate repeated PDE evaluations, and introduce scientific or engineering knowledge into candidate generation [42, 43, 81]. Simulation-guided systems further show how accumulated physical evidence can support the proposal and revision of controller programs or interpretable model structures [82, 83]. However, an LLM-generated design may achieve a strong objective value while violating physical laws, geometric feasibility, conservation requirements, or operating constraints that are not fully represented in the evaluation procedure. This risk is particularly serious when the evaluator is a surrogate model, because the search may exploit its weaknesses and produce apparently high-performing but physically invalid candidates. These advances must therefore remain coupled to high-fidelity numerical and physical validation [80].

Progress should therefore be judged by more than task completion. As discussed in Section 6, executability, numerical accuracy, physical validity, long-horizon reliability, and expert burden cap-

ture different aspects of workflow quality. A useful system should reduce manual effort without concealing modelling assumptions, numerical failures, invalid intermediate decisions, or insufficient evidence. It should also expose the information needed for inspection, including problem specifications, generated code, solver outputs, diagnostics, rejected candidates, and revision decisions. Trustworthy assistance requires not only producing a result, but also preserving the evidence needed to assess how that result was obtained and whether it supports the claimed conclusion.

LLMs will not make numerical expertise obsolete. A more plausible outcome is a redistribution of expert effort. Existing systems already reduce work in solver configuration, numerical-code generation, error correction, learned-solver construction, and optimization-loop coordination [39, 62, 81]. With further progress, experts may spend less time translating intent into routine code, repairing standard software failures, and manually coordinating computational procedures. Their attention can instead shift towards selecting credible models, challenging assumptions, designing informative tests, and judging whether conclusions are scientifically justified. The long-term promise of LLMs in PDE-centred workflows is therefore not the disappearance of expertise, but its concentration at the points where scientific judgement has the greatest value.

8 Conclusion

This Review has organized the emerging literature on LLMs in PDE-centred scientific computing around three workflow stages: Discovery, Solving, and Optimization. In the Discovery Stage, LLMs assist equation discovery, problem formulation, and scientific reasoning. In the Solving Stage, they support solver configuration, numerical-code generation, failure diagnosis, and learned-solver construction. In the Optimization Stage, they connect PDE computation with downstream decisions by formalizing objectives and constraints, coordinating repeated solver evaluations, and enriching candidate generation with scientific and engineering knowledge.

The central conclusion is that the most credible contribution of LLMs to PDE research lies in supporting and coordinating the workflow as a whole, rather than serving as an isolated component within it. Their value comes from connecting natural-language intent, mathematical representations, scientific knowledge, code, computational tools, and feedback across multiple stages of PDE-centred computation. Current systems still depend strongly on expert-defined models, numerical methods, software interfaces, objectives, constraints, and validation criteria, but they already demonstrate the potential to reduce repetitive formulation, implementation, coordination, and repair work. The long-term promise of LLMs is therefore to enable PDE workflows that are more accessible, adaptive, inspectable, and scientifically reliable, while allowing domain experts to focus on modelling choices, numerical validity, and scientific judgement.

References

- [1] Lawrence C. Evans. *Partial Differential Equations*, volume 19 of *Graduate Studies in Mathematics*. American Mathematical Society, Providence, RI, 2 edition, 2010.
- [2] K. W. Morton and D. F. Mayers. *Numerical Solution of Partial Differential Equations: An Introduction*. Cambridge University Press, 2 edition, 2005.
- [3] William L. Oberkampf and Timothy G. Trucano. Verification and validation in computational fluid dynamics. *Progress in Aerospace Sciences*, 38(3):209–272, 2002.

- [4] Lennart Ljung. *System Identification: Theory for the User*. Prentice Hall, Upper Saddle River, NJ, 2 edition, 1999.
- [5] Randall J. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2007.
- [6] H. K. Versteeg and W. Malalasekera. *An Introduction to Computational Fluid Dynamics: The Finite Volume Method*. Pearson Education Limited, Harlow, England, 2 edition, 2007.
- [7] Susanne C. Brenner and L. Ridgway Scott. *The Mathematical Theory of Finite Element Methods*, volume 15 of *Texts in Applied Mathematics*. Springer, New York, 3 edition, 2008.
- [8] Lloyd N. Trefethen. *Spectral Methods in MATLAB*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2000.
- [9] Paola F. Antonietti, Andrea Cangiani, Jennifer Collis, Zhaonan Dong, Emmanuil H. Georgoulis, Stefano Giani, and Paul Houston. Review of discontinuous galerkin finite element methods for partial differential equations on complicated domains. In *Building Bridges: Connections and Challenges in Modern Approaches to Numerical Partial Differential Equations*, volume 114 of *Lecture Notes in Computational Science and Engineering*, pages 279–308. Springer, Cham, 2016.
- [10] Guglielmo Vivarelli, Ning Qin, and Shahrokh Shahpar. A review of mesh adaptation technology applied to computational fluid dynamics. *Fluids*, 10(5):129, 2025.
- [11] Jacques Louis Lions. *Optimal Control of Systems Governed by Partial Differential Equations*, volume 170 of *Grundlehren der mathematischen Wissenschaften*. Springer, Berlin, Heidelberg, 1971.
- [12] Michael Hinze, Rene Pinnau, Michael Ulbrich, and Stefan Ulbrich. *Optimization with PDE Constraints*, volume 23 of *Mathematical Modelling: Theory and Applications*. Springer, Dordrecht, 2009.
- [13] Max D. Gunzburger. *Perspectives in Flow Control and Optimization*. Advances in Design and Control. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2002.
- [14] Martin P. Bendsøe and Ole Sigmund. *Topology Optimization: Theory, Methods, and Applications*. Springer, Berlin, Heidelberg, 2 edition, 2004.
- [15] Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 113(15):3932–3937, 2016.
- [16] Samuel H. Rudy, Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Data-driven discovery of partial differential equations. *Science Advances*, 3(4):e1602614, 2017.
- [17] Laure Zanna and Thomas Bolton. Data-driven equation discovery of ocean mesoscale closures. *Geophysical Research Letters*, 47(17):e2020GL088376, 2020.
- [18] Sarah Beetham and Jesse Capecehatro. Formulating turbulence closures using sparse regression with embedded form invariance. *Physical Review Fluids*, 5(8):084611, 2020.

- [19] Joseph Bakarji and Daniel M Tartakovsky. Data-driven discovery of coarse-grained equations. *Journal of Computational Physics*, 434:110219, 2021.
- [20] Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- [21] Yohai Bar-Sinai, Stephan Hoyer, Jason Hickey, and Michael P. Brenner. Learning data-driven discretizations for partial differential equations. *Proceedings of the National Academy of Sciences*, 116(31):15344–15349, 2019.
- [22] Dmitrii Kochkov, Jamie A. Smith, Ayya Alieva, Qing Wang, Michael P. Brenner, and Stephan Hoyer. Machine learning-accelerated computational fluid dynamics. *Proceedings of the National Academy of Sciences*, 118(21):e2101784118, 2021.
- [23] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nat. Mach. Intell.*, 3(3):218–229, 2021.
- [24] Zongyi Li, Nikola Borislavov Kovachki, Kamyar Azizzadenesheli, Burigede liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.
- [25] Alasdair Tran, Alexander Mathews, Lexing Xie, and Cheng Soon Ong. Factorized Fourier neural operators. In *International Conference on Learning Representations*, 2023.
- [26] Lu Lu, Raphael Pestourie, Wenjie Yao, Zhicheng Wang, Francesc Verdugo, and Steven G Johnson. Physics-informed neural networks with hard constraints for inverse design. *SIAM Journal on Scientific Computing*, 43(6):B1105–B1132, 2021.
- [27] Jeremy Yu, Lu Lu, Xuhui Meng, and George Em Karniadakis. Gradient-enhanced physics-informed neural networks for forward and inverse pde problems. *Computer Methods in Applied Mechanics and Engineering*, 393:114823, 2022.
- [28] Chengping Rao, Pu Ren, Qi Wang, Oral Buyukozturk, Hao Sun, and Yang Liu. Encoding physics to learn reaction–diffusion processes. *Nature Machine Intelligence*, 5(7):765–779, 2023.
- [29] Han Wan, Rui Zhang, Qi Wang, Yang Liu, and Hao Sun. Pesanet: Physics-encoded spectral attention network for simulating pde-governed complex systems. In *International Joint Conference on Artificial Intelligence*, pages 1–7, 2025.
- [30] Jean Rabault, Miroslav Kuchta, Atle Jensen, Ulysse Réglade, and Nicolas Cerardi. Artificial neural networks trained through deep reinforcement learning discover control strategies for active flow control. *Journal of Fluid Mechanics*, 865:281–302, 2019.
- [31] Philipp Holl, Nils Thuerey, and Vladlen Koltun. Learning to control pdes with differentiable physics, 2020.
- [32] Jonathan Viquerat, Jean Rabault, Alexander Kuhnle, Hassan Ghraieb, Aurélien Larcher, and Elie Hachem. Direct shape optimization through deep reinforcement learning. *Journal of Computational Physics*, 428:110080, 2021.

- [33] Kelsey Allen, Tatiana Lopez-Guevara, Kimberly L. Stachenfeld, Alvaro Sanchez-Gonzalez, Peter Battaglia, Jessica B. Hamrick, and Tobias Pfaff. Inverse design for fluid-structure interactions using graph network simulators. In *Advances in Neural Information Processing Systems*, volume 35, pages 13759–13774, 2022.
- [34] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.
- [35] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS ’22, Red Hook, NY, USA, 2022. Curran Associates Inc.
- [36] Ekin Karpas, Yoav Levine, Sabrina J. Mielke, Yonatan Belinkov, Barak Lenz, Omer Lieber, et al. Mrkl systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning. *arXiv preprint arXiv:2205.00445*, 2022.
- [37] Mengge Du, Yuntian Chen, Zhongzheng Wang, Longfeng Nie, and Dongxiao Zhang. Llm4ed: Large language models for automatic equation discovery. *arXiv preprint arXiv:2405.07761*, 2024.
- [38] Mingquan Feng, Yixin Huang, Yizhou Liu, Bofang Jiang, and Junchi Yan. Physpde: Rethinking pde discovery and a physical hypothesis selection benchmark. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [39] Shanda Li, Tanya Marwah, Junhong Shen, Weiwei Sun, Andrej vektor Risteski, Yiming Yang, and Ameet Talwalkar. Codepde: An inference framework for llm-driven pde solver generation. *Transactions on Machine Learning Research*, 2026. Accepted by TMLR.
- [40] Ling Yue, Nithin Somasekharan, Yadi Cao, and Shaowu Pan. Foam-agent: Towards automated intelligent cfd workflows. 2025.
- [41] Shaghayegh Fazliani and Madeleine Udell. Pde-sharp: Pde solver hybrids through analysis and refinement passes. *arXiv preprint arXiv:2511.00183*, 2025.
- [42] Mauricio Soroco, Jialin Song, Mengzhou Xia, Kye Emond, Weiran Sun, and Wuyang Chen. Pde-controller: Llms for autoformalization and reasoning of pdes. In *International Conference on Machine Learning*. PMLR, 2025.
- [43] Xinxin Zhang, Zhuoqun Xu, Guangpu Zhu, Chien Ming Jonathan Tay, Yongdong Cui, Boo Cheong Khoo, and Lailai Zhu. Using large language models for parametric shape optimization. *Physics of Fluids*, 37(8):083601, 2025.

- [44] JuraJ Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Petar Sirkovic, Artiom Myaskovsky, Grzegorz Glowaty, Felix Weissenberger, Alessio Orlandi, Dan Popovici, et al. Accelerating scientific discovery with co-scientist. *Nature*, 655:487–496, 2026.
- [45] Ali E. Ghareeb, Benjamin Chang, Ludovico Mitchener, Angela Yiu, Caralyn J. Szostkiewicz, Dmytro Shved, Gavin J. Gyimesi, Jon M. Laurent, Samantha M. Wright, Muhammed T. Razzak, Andrew D. White, Silvia C. Finnemann, Michaela M. Hinks, and Samuel G. Rodrigues. A multi-agent system for automating scientific discovery. *Nature*, 655:497–505, 2026.
- [46] Kai Ruan, Xuan Wang, Jixiang Hong, Peng Wang, Yang Liu, and Hao Sun. Evaluating llms’ divergent thinking capabilities for scientific idea generation with minimal context. *Nature Communications*, 17(1):3625, 2026.
- [47] Agah Drajat Garnadi. Llm assisted mathematical modeling: Homogeneous laplace equation in cylinder with the complete electrode model. 2025.
- [48] Zongmin Yu and Liu Yang. Agentic symbolic search: Characterizing pdes beyond hand-crafted expressions, meshes, and neural networks, 2026.
- [49] Microsoft Research AI4Science and Microsoft Azure Quantum. The impact of large language models on scientific discovery: a preliminary study using gpt-4. 2023.
- [50] Runxiang Wang, Boxiao Wang, Kai Li, Yifan Zhang, and Jian Cheng. Drsrr: Llm based scientific equation discovery with dual reasoning from data and experience. *arXiv preprint arXiv:2506.04282*, 2025.
- [51] Parshin Shojaee, Kazem Meidani, Shashank Gupta, Amir Barati Farimani, and Chandan K Reddy. Llm-sr: Scientific equation discovery via programming with large language models. *arXiv preprint arXiv:2404.18400*, 2024.
- [52] Rohan Bhatnagar, Ling Liang, Krish Patel, and Haizhao Yang. From equations to insights: Unraveling symbolic structures in pdes with llms. *arXiv preprint arXiv:2503.09986*, 2025.
- [53] Pingchuan Ma, Tsun-Hsuan Wang, Minghao Guo, Zhiqing Sun, Joshua B Tenenbaum, Daniela Rus, Chuang Gan, and Wojciech Matusik. Llm and simulation as bilevel optimizers: A new paradigm to advance physical scientific discovery. *arXiv preprint arXiv:2405.09783*, 2024.
- [54] Jiachen Guo, Chanwook Park, Dong Qian, Thomas J. R. Hughes, and Wing Kam Liu. Large language model-empowered next-generation computer-aided engineering. 2025.
- [55] Zhen Hang, Yushan Yashengjiang, Junhui Li, Huanshuo Dong, Yang Wei, Zhezheng Hao, Jiangtao Ma, Songlin Bai, Haozhong Kai, Xihang Yue, Gangzong Si, Dongming Jiang, Chao Yao, Zhanhua Hu, Jiangqing Zhang, Pengwei Liu, Yaomin Shen, Xingyu Ren, Lei Liu, Zikang Xu, Han Li, Qingsong Yao, Hande Dong, and Hong Wang. Pdeagent-bench: A multi-metric, multi-library benchmark for pde solver generation, 2026.
- [56] Qile Jiang, Zhiwei Gao, and George Em Karniadakis. Deepseek vs. chatgpt vs. claude: A comparative study for scientific computing and scientific machine learning tasks. *Theoretical and Applied Mechanics Letters*, 15(3):100583, 2025.
- [57] Rushikesh Deotale, Adithya Srinivasan, Mahmoud Golestanian, Yuan Tian, Tianyi Zhang, Pavlos Vlachos, and Hector Gomez. All-fem: Agentic large language models fine-tuned for

- finite element methods. *Computer Methods in Applied Mechanics and Engineering*, 457:118985, 2026.
- [58] Haoyang Wu, Xinxin Zhang, and Lailai Zhu. Automated code development for pde solvers using large language models. 2025.
 - [59] Jianda Du, Youran Sun, and Haizhao Yang. Autonumerics: An autonomous, pde-agnostic multi-agent pipeline for scientific computing, 2026.
 - [60] Long Wang, Lei Zhang, and Guowei He. Evaluations of large language models in computational fluid dynamics: Leveraging, learning and creating knowledge. *Theoretical and Applied Mechanics Letters*, 15(3):100597, 2025.
 - [61] Nithin Somasekharan, Ling Yue, Yadi Cao, Weichao Li, Patrick Emami, Pochinapeddi Sai Bhargav, Anurag Acharya, Xingyu Xie, and Shaowu Pan. Cfdllmbench: A benchmark suite for evaluating large language models in computational fluid dynamics. 2025.
 - [62] Sandeep Pandey, Ran Xu, Wenkang Wang, and Xu Chu. Openfoamgpt: A retrieval-augmented large language model (llm) agent for openfoam-based computational fluid dynamics. *Physics of Fluids*, 37(3), 2025.
 - [63] Nithin Somasekharan, Rabi Pathak, Manushri Dhanakoti, Tingwen Zhang, Ling Yue, Andy Zhu, and Shaowu Pan. Ai cfd scientist: Toward open-ended computational fluid dynamics discovery with physics-aware ai agents, 2026.
 - [64] Jingsen Feng, Ran Xu, and Xu Chu. Openfoamgpt 2.0: End-to-end, trustworthy automation for computational fluid dynamics. *International Journal of Heat and Fluid Flow*, 120:110399, 2026.
 - [65] Yuxuan Chen, Xu Zhu, Hua Zhou, and Zhuyin Ren. Metaopenfoam: an llm-based multi-agent framework for cfd. 2024.
 - [66] Yuxuan Chen, Xu Zhu, Hua Zhou, and Zhuyin Ren. Metaopenfoam 2.0: Large language model driven chain of thought for automating cfd simulation and post-processing. *Journal Name*, 2025.
 - [67] E Fan, Kang Hu, Zhuowen Wu, Jiangyang Ge, Jiawei Miao, Yuzhi Zhang, He Sun, Weizong Wang, and Tianhan Zhang. Chatcfd: An llm-driven agent for end-to-end cfd automation with domain-specific structured reasoning. 2025.
 - [68] Zhehao Dong, Zhen Lu, and Yue Yang. Fine-tuning a large language model for automating computational fluid dynamics simulations. *Theoretical and Applied Mechanics Letters*, 15:100594, 2025.
 - [69] Mohamad Ali-Dib and Kristen Menou. Physics simulation capabilities of llms. *Physica Scripta*, 99(11):116003, oct 2024.
 - [70] Varun Kumar, Leonard Gleyzer, Adar Kahana, Khemraj Shukla, and George Karniadakis. Mycrunchgpt: A chatgpt assisted framework for scientific machine learning. *Journal of Machine Learning for Modeling and Computing*, 4(4):41–72, January 2023.
 - [71] Qingpo Wuwu, Chonghan Gao, Tianyu Chen, Yihang Huang, Yuekai Zhang, Jianing Wang, Jianxin Li, Haoyi Zhou, and Shanghang Zhang. Pinnsagent: Automated pde surrogation with large language models. *arXiv preprint arXiv:2501.12053*, 2025.

- [72] Xin He, Liangliang You, Hongduan Tian, Bo Han, Ivor Tsang, and Yew-Soon Ong. Lang-pinn: From language to physics-informed neural networks via a multi-agent framework. 2025.
- [73] Jiajun Bao, Nicolas Boullé, Toni JB Liu, Raphaël Sarfati, and Christopher J Earls. Text-trained llms can zero-shot extrapolate pde dynamics. *arXiv preprint arXiv:2509.06322*, 2025.
- [74] Hang Zhou, Yuezhou Ma, Haixu Wu, Haowen Wang, and Mingsheng Long. Unisolver: Pde-conditional transformers towards universal neural pde solvers. In *Forty-second International Conference on Machine Learning*, 2025.
- [75] Junhong Shen, Tanya Marwah, and Ameet Talwalkar. UPS: Efficiently building foundation models for PDE solving via cross-modal adaptation. *Transactions on Machine Learning Research*, 2024.
- [76] Max Zhu, Adrián Bazaga, and Pietro Liò. Fluid-llm: Learning computational fluid dynamics with spatiotemporal-aware large language models. 2024.
- [77] Anthony Zhou, Zijie Li, Michael Schneier, John R. Buchanan, and Amir Barati Farimani. Text2PDE: Latent diffusion models for accessible physics simulation. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- [78] Steve D. Yang, Zulfikhar A. Ali, and Bryan M. Wong. Fluid-gpt (fast learning to understand and investigate dynamics with a generative pre-trained transformer): Efficient predictions of particle trajectories and erosion. *Industrial & Engineering Chemistry Research*, 62(37):15278–15289, 2023.
- [79] Ye Liu, Shouyi Liu, Huiyu Yang, Jiangang Gu, Wenhao Fan, Zhongxin Yang, Ding Wang, Simeng Chen, Zirun Jiang, Yuanwei Bin, Shiyi Chen, and Yuntian Chen. Aeroagent: A vision-physics-decision framework for aerodynamic vehicle design. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11694–11703, June 2026.
- [80] Shaghayegh Fazliani, Krissh Chawla, Jack Guo, Yiren Shen, Matthias Ihme, and Madeleine Udell. Shapebench: A scalable benchmark and diagnostic suite for standardized evaluation in aerodynamic shape optimization, 2026.
- [81] Yuxuan Chen, Long Zhang, Xu Zhu, Hua Zhou, and Zhuyin Ren. Optmetaopenfoam: Large language model driven chain of thought for sensitivity analysis and parameter optimization based on cfd. *Journal Name*, 2025.
- [82] Boai Sun, Wenjin Guo, Zongmin Yu, and Liu Yang. Self-evolving scientific agent discovers generalizable physically-reasoned fluid control, 2026.
- [83] Zhongxin Yang, Yuanwei Bin, Yipeng Shi, and Xiang I. A. Yang. Large language model driven development of turbulence models. 2025.
- [84] Hugon Lee, Hyeonbin Moon, Junhyeong Lee, and Seunghwa Ryu. Toward knowledge-guided ai for inverse design in manufacturing: A perspective on domain, physics, and human-ai synergy. *Advanced Intelligent Discovery*, 2025.
- [85] Varun Kumar and George Em Karniadakis. Toward autonomous engineering design: A knowledge-guided multi-agent framework, 2025.

- [86] Jianke Yang, Ohm Venkatachalam, Mohammad Kianezhad, Sharvaree Vadgama, and Rose Yu. Think like a scientist: Physics-guided llm agent for equation discovery, 2026.
- [87] Hao Xu, Yuntian Chen, Rui Cao, Tianning Tang, Mengge Du, Jian Li, Adrian H. Callaghan, and Dongxiao Zhang. Generative discovery of partial differential equations by learning from math handbooks, 2025.
- [88] Elisa Negrini, Yuxuan Liu, Liu Yang, Stanley J. Osher, and Hayden Schaeffer. A multimodal pde foundation model for prediction and scientific text descriptions. 2025.
- [89] H. Jasak, A. Jemcov, Z. Tukovic, et al. Openfoam: A C++ library for complex physics simulations. In *International Workshop on Coupled Methods in Numerical Dynamics*, volume 1000, pages 1–20, Dubrovnik, Croatia, 2007.
- [90] Makoto Takamoto, Timothy Praditia, Raphael Leiteritz, Dan MacKinlay, Francesco Alesiani, Dirk Pflüger, and Mathias Niepert. PDEBench: An Extensive Benchmark for Scientific Machine Learning. In *36th Conference on Neural Information Processing Systems (NeurIPS 2022) Track on Datasets and Benchmarks*, 2022.
- [91] Zhongkai Hao, Jiachen Yao, Chang Su, Hang Su, Ziao Wang, Fanzhi Lu, Zeyu Xia, Yichi Zhang, Songming Liu, Lu Lu, and Jun Zhu. Pinnacle: a comprehensive benchmark of physics-informed neural networks for solving pdes. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, Red Hook, NY, USA, 2024. Curran Associates Inc.